

TiLiA – TimeLineAnnotator

Felipe D. Martins^{1*}, Anne R. E. Foo^{2*}, and Mark R. H. Gotham^{3*}

¹ Universidade Federal do Rio de Janeiro, Brazil ² École Polytechnique Fédérale de Lausanne, Digital and Cognitive Musicology Lab, Switzerland ³ King's College London, United Kingdom * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: [Open Journals](#) 

Reviewers:

- [@openjournals](#)

Submitted: 01 January 1970

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#))

Summary

'TiLiA' (TimeLine Annotator) is an open-source, cross-platform network of tools and resources designed to facilitate real-time creating, displaying and interacting with timeline-style annotations of audio, video and musical scores. Developed in Python with the PySide UI framework, TiLiA is available primarily as a desktop Graphical User Interface (GUI), supported by a Command Line Interface (CLI) as well as a free and open online platform where users may upload, view and query existing analyses.

Statement of need

Visual analyses of music have proven to be an enduringly popular, effective, and versatile tool for conveying a variety of musical ideas.¹ As the famous adage goes: 'a picture is worth a thousand words'. These analytical images are primarily connected to the *analysis* rather than the *music*, thus making them readily useful not only for the already visual sources of notated music, but also for the much broader range of musical styles and repertoires that are transmitted without the use of notation.

Whatever potential a *static* image has, an *interactive* one offers considerably more. Formal analysis is one of the clearest beneficiaries of the visual summary of music. Form can be hard to parse (especially in real-time) but is often easy to understand in relatively simple, at-a-glance summaries.

Save for a relatively small early body of literature², musical scholarship has not yet made the leap into this technological space with any degree of decisiveness or consistency. Even most visually minded music analyses are consigned to static images embedded in print (or in e-copies at best) and not subject to the expectations for *fair*, open-source data sharing that are now *de rigueur* in other fields. Still among the more computationally inclined communities, there has been little progress made on the coordination of standards and corpora for formal analysis.

In today's digital age, we have come to expect certain basic interactive features across visual interfaces, such as **resizing** content to fit the window, keyboard shortcuts, undo/redo functionality, and so on. Musical contexts extend this with additional expectations, such as **real-time synchronisation** (a.k.a., audio or score following). Any tool that aims to win over users effectively must present analytical ideas naturally and intuitively, with minimal friction in the user interface.

While one *can* appropriate tools (which primary purpose is editing scores or audio) for creating

¹See Isaacson (2023) for a survey of this long history.

²Gotham & Ireland (2019), for instance, is a first step towards a standard for formal annotations. Balke et al. (2017) and Rosenzweig et al. (2020) provide interfaces for interacting with specific annotation corpora. See also footnotes below.

37 visual analyses,^{3 4} the current range of tools dedicated to the task typically offer limited
38 functionality and their uptake by the wider community has been minimal.⁵

39 In response to these challenges (user-friendly design, interoperable formats, real-time
40 interactivity ...), we present 'TiLiA': a timeline annotator for all.

41 Specifications

42 TiLiA consists of an organisation of digital tools, primarily of its [open-source desktop application](#)
43 [and command-line interface](#), but also of a supporting online platform. The desktop application
44 is developed in Python using the PySide binding for the Qt UI framework. Automated testing
45 is done with pytest, and fully-automated deployment is available via GitHub Actions. The
46 code base is loosely organised around a Model-View-Controller (MVC) pattern, which allows
47 both the CLI and the GUI to benefit from the same backend logic. We provide object-oriented
48 base classes such as Timeline, TimelineUI and TimelineComponent as a means to promote
49 extensibility and support an evergrowing range of annotation needs.

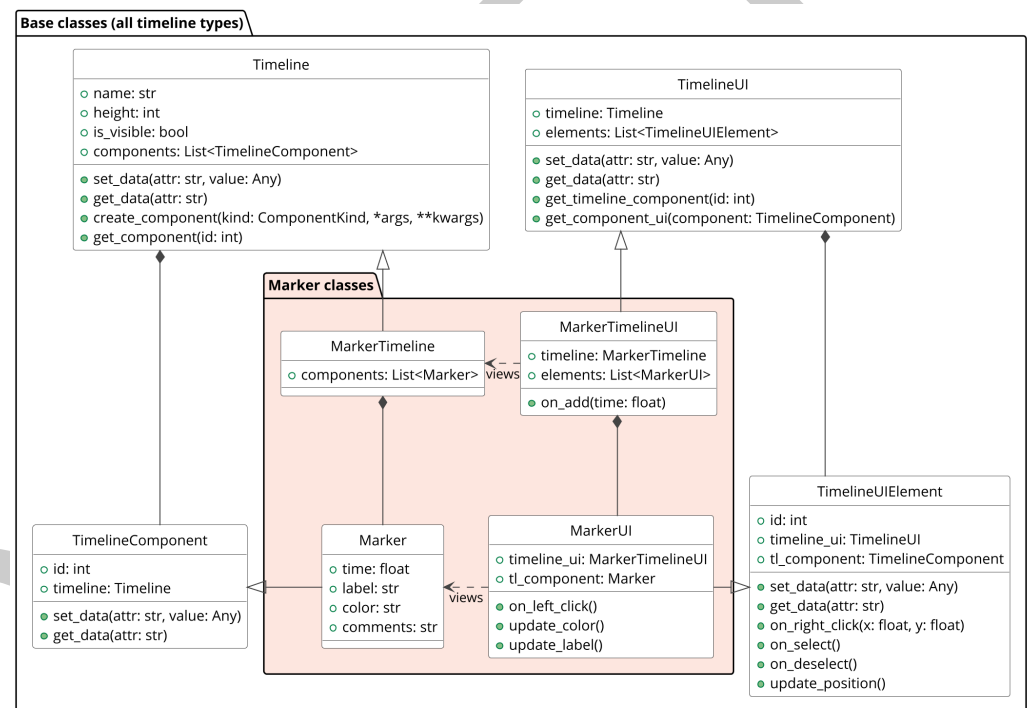


Figure 1: A simplified diagram for the main timeline base classes.

50 Figure 1 is a simplified diagram for the main timeline base classes. The outer box shows the base
51 classes shared by all timeline types; the inner box shows the concrete subclasses for the marker

³The music notation software *Sibelius*, for instance, has offered a timeline since c.2019 with bar numbers and rudimentary checkpoints for changes to time and key signatures, presumably because they are possible indicators of section breaks. Peebles (2013) discusses how to use an audio editor (*Audacity*) in a similar way to the tools discussed here.

⁴Additionally, websites like *Edirom Online* (Röwenstrunk, 2014), *Listen Here!* (Weigl et al., 2023) and *mei-friend* (Goehl & Weigl, 2024) support the digital annotation of scores, not specific to analysis.

⁵Yorgason's *Audio Timeliner* (Yorgason, 2018), based on the original *Variations Audio Timeliner* (Notess & Swan, 2004), is notable for capturing the needs of its users with its simple design, but is not open-source. *iAnalyse5* (Coupré, 2019) is a similar closed-source offering that is also not cross-platform. *Dezrann* (Giraud et al., 2018) is a noteworthy, actively maintained and open-source website that allows the alignment of score data on top of annotation. *Sonic Visualiser* (Cannam et al., 2010) excels when it comes to annotating and visualising audio, but lacks integration with musical scores.

52 timeline type, one example among the various timeline types. Timeline acts as container for
53 TimelineComponents. Both act as models of TimelineUI and TimelineUIElement, which are
54 views; in the marker example, MarkerTimelineUI and MarkerUI view MarkerTimeline and
55 Marker respectively.

56 TiLiA is designed to facilitate the *creation* of analyses — particularly where this requires
57 complex alignment with audio/video sources — and the (conversion and) *import* of analyses
58 of external origin through CSV files. This effort thus connects TiLiA to wider datasets and
59 scholarship, while also expanding its functionality. The [tilia-dcml](#) repository demonstrates how
60 the TiLiA CLI might be used to visualise data from an external corpus of musical analyses.
61 The CLI can also *export* timelines created in TiLiA to a JSON file suitable for processing by
62 other software.

63 The desktop application

64 The main part of TiLiA is a cross-platform *What-You-See-Is-What-You-Get* GUI ([Figure 2](#)),
65 designed to feel intuitive to anyone who has worked with audiovisual editing software. A
66 TiLiA analysis consists of timelines, each containing annotations that can be aligned to an
67 audio, video, YouTube stream or musical score. Each *timeline type* is designed to support
68 conventional formal annotations, with the affordance to work with a variety of other analytical
69 styles. These timelines each contain one or more *component types* (e.g., markers for marker
70 timelines), each with different *properties* to convey information including comments, colour and
71 labels.

72 We connect timelines and their components by using the loaded media as a “base layer” to
73 which all timelines refer. Timeline components have either `time` or `start_time` and `end_time`
74 timestamps which position them in relation to the underlying media. If a different media is
75 loaded, the user has the option to either uniformly scale timelines to the new media or to keep
76 them at their original position (deleting out-of-bounds components if necessary). If a beat
77 timeline is present, the software can map timestamps to metric positions, also providing a
78 metric-based interface for importing, exporting and interacting with timelines.

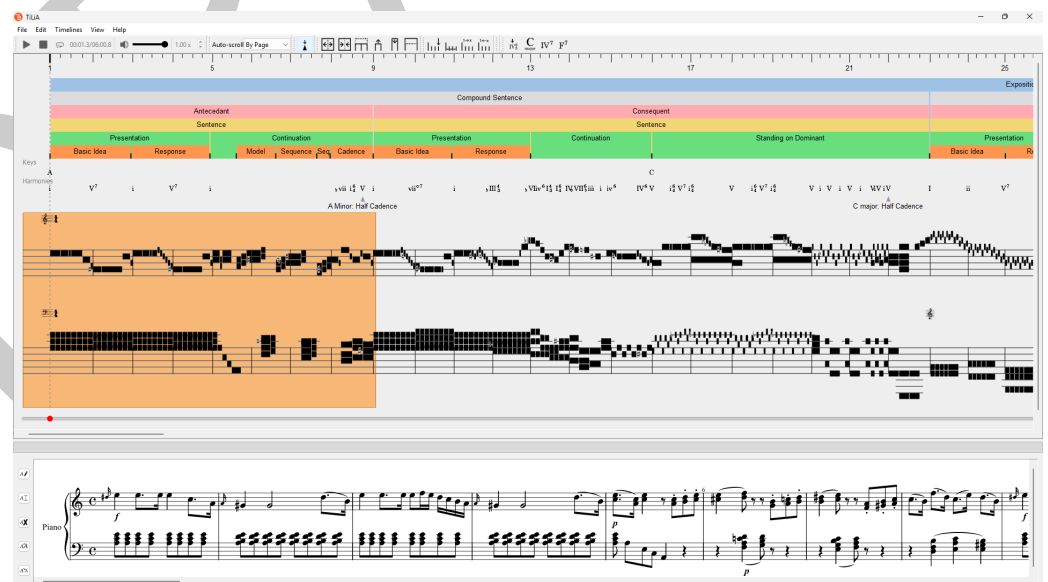


Figure 2: Excerpt of a TiLiA analysis on the desktop application.

79 Currently, there are eight types of timelines:

Timeline type	Purpose
Audiowave	Displaying amplitude graphs
Beat	Marking beats/measures, synchronising metrical and absolute time
Harmony	Chord symbols and roman numeral analysis
Hierarchy	Representing hierarchical relations and sequences
Marker	Annotating discrete timepoints
PDF	Synchronising external PDF documents with playback
Range	Representing non-hierarchical relations and sequences
Score	Synchronising digitised scores with playback

Command-line interface

In addition to the desktop GUI and the web app, a **command line interface** allows the automation of file and timeline creation and edition, media loading and external data importing. Sequences of commands can be loaded by the CLI from scripts written in plain text files, enabling programmatic conversion from various data formats to TiLiA files.

```

metadata set-media-length 120
timelines add hierarchy --name "Form"
timelines add marker --name "Formal closures"
timelines add hierarchy --name "Harmonic segments"
timelines add beat --name "Measures" --beat-pattern 1
timelines add marker --name "Harmony"
timelines add marker --name "Harm. functions"
timelines add marker --name "Keys"

load-media <path to audio> --scale-timelines yes
metadata set title "Title"
save <path to TiLiA file> --overwrite
export <path to JSON file> --overwrite

```

Online platform

The TiLiA desktop application is supported by an **online platform** that lets users store, share, visualise and even query existing analyses. Registration is free and is only required for uploading content. This reflects a goal of TiLiA to support not only the modernised creation and storage of analyses, but also to support the still-rare practice of analytical *collaboration* and structured *searches* on the corpora; both which benefit from network effects facilitated by the lowered entry barrier.

Outlook

The TiLiA desktop application aims to provide a user-friendly annotation tool both for MIR researchers and musicologists. It can be used to create large annotation datasets or detailed, interactive analyses of individual pieces. It can also be used in the classroom to better connect theoretical concepts with audiovisual material or as a common interface for analytical assignments.

In the future we hope to convert existing open-source datasets and integrate them into the web platform, providing a centralised hub where diversified audiovisual data can be queried and interacted with in a uniform manner. We also plan to improve audio processing and score displaying capabilities and to add more timeline types (e.g. a 2-dimensional graph timeline) to offer more diversified visualisations.

References

- Balke, S., Hiemer, M., Schwab, P., Arifi-Müller, V., Meyer-Wegener, K., & Müller, M. (2017). Die Oper als Multimediaszenario – Wagners Walküren gehen online. *INFORMATIK 2017*, 75–86. https://doi.org/10.18420/in2017_04
- Cannam, C., Landone, C., & Sandler, M. (2010). Sonic Visualiser: An Open Source Application for Viewing, Analysing, and Annotating Music Audio Files. *Proceedings of the 18th ACM International Conference on Multimedia*. <https://doi.org/10.1145/1873951.1874248>
- Couprie, P. (2019). iAnalyse 5 : Le développement d'un logiciel pour la musicologie numérique. *Journées d'informatique Musicale (JIM 2019)*. <https://hal.science/hal-02133538>
- Giraud, M., Groult, R., & Leguy, E. (2018). Dezzrann, a Web Framework to Share Music Analysis. In S. Bhagwati & J. Bresson (Eds.), *Proceedings of the International Conference on Technologies for Music Notation and Representation – TENOR'18* (pp. 104–110). Concordia University. ISBN: 978-1-5251-0551-7
- Goebl, W., & Weigl, D. M. (2024). Mei-friend: An interactive web-based editor for digital music encodings. *Journal of Open Source Software*, 9(98), 6002. <https://doi.org/10.21105/joss.06002>
- Gotham, M. R. H., & Ireland, M. (2019). Taking Form: A Representation Standard, Conversion Code, and Example Corpora for Recording, Visualizing, and Studying Analyses of Musical Form. In A. Flexer, G. Peeters, J. Urbano, & A. Volk (Eds.), *Proceedings of the 20th International Society for Music Information Retrieval Conference, ISMIR 2019, Delft, The Netherlands, November 4-8, 2019* (pp. 693–699). <http://archives.ismir.net/ismir2019/paper/000084.pdf>
- Isaacson, E. (2023). *Visualizing Music*. Indiana University Press. <https://www.barnesandnoble.com/w/visualizing-music-eric-isaacson/1140906228?ean=9780253064738>
- Notess, M., & Swan, M. (2004). Timeliner: Building a learning tool into a digital music library. In L. Cantoni & C. McLoughlin (Eds.), *Proceedings of EdMedia + innovate learning 2004* (pp. 603–609). Association for the Advancement of Computing in Education (AACE). <https://www.learntechlib.org/p/12995>
- Peebles, C. (2013). Using Audacity to Participate in Active Musical Listening. *Engaging Students: Essays in Music Pedagogy*. <http://flipcamp.org/engagingstudents/peebles.html>
- Rosenzweig, S., Scherbaum, F., Shugliashvili, D., Arifi-Müller, V., & Müller, M. (2020). Erkomaishvili Dataset: A Curated Corpus of Traditional Georgian Vocal Music for Computational Musicology. *Transactions of the International Society for Music Information Retrieval*, 3(1), 31–41. <https://doi.org/10.5334/tismir.44>
- Röwenstrunk, D. (2014). *Markieren, Codieren, Annotieren – Arbeiten an und mit digitalen Musikeditionen mit Edirom und MEI*.
- Weigl, D. M., VanderHart, C., Rammler, D., Pescoller, M., & Goebl, W. (2023). Listen here! A web-native digital musicology environment for machine-assisted close listening. *Proceedings of the 10th International Conference on Digital Libraries for Musicology*, 109–118. <https://doi.org/10.1145/3625135.3625144>
- Yorgason, B. (2018). Teaching with Timelines. *Engaging Students*, 6. <http://flipcamp.org/engagingstudents6/essays/yorgason.html>